

Texte de soutenance — Noa Orand (E6)

Script oral du diaporama Diaporama_soutenance_Noa_FUSION.pptx (31 slides).

Registre parlé, à la première personne. Durée cible : ~18-20 min de présentation, puis questions.

Repères de minutage cumulés indiqués entre crochets. À adapter à ton débit en répétant.

Slide 1 — Titre

[0:00]

Bonjour, je m'appelle Noa Orand, je suis en BTS CIEL option Informatique et Réseaux. Je vais vous présenter notre projet de fin d'études : l'estimation, par intelligence artificielle, du niveau de tension d'un moteur électrique, à partir de deux signaux physiques — le son et la vibration. Le projet a été mené à quatre ; je présenterai d'abord le contexte commun, puis ma contribution personnelle.

Slide 2 — Sommaire

[0:30]

Ma présentation suit quatre temps : d'abord la présentation générale du projet, ensuite la partie physique — le banc, les capteurs, l'acquisition —, puis ma contribution individuelle sur le logiciel d'acquisition et le modèle d'IA, et enfin la conclusion avec nos résultats et nos perspectives.

Slide 3 — Partie 1 : Présentation du projet [1:00]

Commençons par le cadre du projet : le contexte académique, l'enjeu industriel, et le choix du sujet.

Slide 4 — Cadre académique et équipe

[1:10]

Ce projet s'inscrit dans l'épreuve E6 du BTS CIEL, session 2026. Il a mobilisé l'ensemble des compétences de la formation : programmation système, réseaux, électronique embarquée, et intelligence artificielle. Nous étions une équipe de quatre. Pour ma part, j'avais en charge le pipeline d'acquisition logicielle et le modèle d'IA ; Hugo s'occupait de la qualité des données et du versioning, Mederick du firmware ESP32 et de la capture audio, et Ege de l'IHM Qt et de la supervision des captures.

Slide 5 — Maintenance prédictive : l'enjeu industriel

[1:45]

L'idée de départ vient de la maintenance prédictive. Dans l'industrie, écouter et mesurer les vibrations d'une machine permet de détecter une dérive **avant** la panne, et donc d'éviter les arrêts non planifiés. C'est une analyse non intrusive et riche en information. On s'est appuyés sur des jeux de données de référence du domaine — MIMII, CWRU, MAFAULDA — qui montrent qu'un réseau de neurones convolutif sur mel-spectrogramme apprend très efficacement ces signatures. Nous captions le son à 16 kHz et la vibration autour de 10 Hz effectifs.

Slide 6 — Pivot du projet : du CNC au banc moteur

[2:30]

Au départ, le sujet visait la détection d'usure d'outil sur une machine CNC. Mais l'accès à cette machine était très limité, et nous ne pouvions pas produire un jeu de données suffisant et reproductible. Nous avons donc pris une décision d'ingénierie : recentrer le projet sur un banc moteur électrique. Le banc est reproductible, les conditions sont maîtrisées, le dataset constituable dans le temps imparti — et surtout, la logique du projet est conservée : faire de l'IA sur des signaux physiques. Ce pivot, je le présente comme une décision assumée, pas comme un repli.

Slide 7 — Objectif fonctionnel

[3:15]

L'objectif fonctionnel tient en quatre étapes. **Mesurer** : capter en continu le son et la vibration. **Transmettre** : envoyer les données au PC par Wi-Fi, en UDP pour les signaux et en TCP pour les métadonnées. **Stocker** : constituer un dataset CSV indexé par un fichier maître. Et **Prédire** : un modèle CNN double branche qui sort un pourcentage de la tension maximale. C'est cette chaîne complète que nous avons réalisée.

Slide 8 — Partie 2 : Partie physique [4:00]

J'en viens à la partie physique : le banc d'essai, les capteurs et la chaîne d'acquisition.

Slide 9 — Banc d'essai

[4:10]

Le banc est construit autour d'un moteur à courant continu alimenté par une alimentation à tension réglable, ce qui nous permet de balayer toute la plage de fonctionnement. Nous avons retenu deux conditions de charge pour leur reproductibilité : la condition A, à vide, sans frein, avec une tension maximale mesurée de 30,3 V ; et la condition B, avec frein, à 28,7 V. Dans chaque condition, cette tension maximale sert de référence à 100 %. Une troisième condition, avec frein et inertie, a été écartée : nous ne pouvions pas garantir un couple reproductible d'une session à l'autre.

Slide 10 — Capteurs et chaîne d'acquisition

[5:00]

Côté son, nous utilisons un microphone MEMS numérique I2S, l'INMP441 : 16 kHz théoriques, environ 15,8 kHz effectifs après optimisation du firmware, en mono sur 32 bits. Côté vibration, un capteur WitMotion WTVB01-BT50, en Bluetooth Low Energy : il envoie des trames de 28 octets contenant treize valeurs — vitesses, angles, déplacements, fréquences sur trois axes. Il est annoncé à 200 Hz, mais on tombe à environ 10 Hz effectifs à cause d'un conflit radio entre le Bluetooth et le Wi-Fi qui partagent la même antenne sur l'ESP32.

Slide 11 — Théorème de Shannon

[5:45]

Cette histoire de fréquences m'amène à un fondement théorique : le théorème de Shannon. La question est : combien de fois par seconde faut-il mesurer un signal pour pouvoir le reconstituer sans perte ? Le théorème de Nyquist-Shannon répond : la fréquence d'échantillonnage doit valoir au moins le double de la fréquence maximale utile du signal — c'est la formule **f-e supérieur ou égal à deux fois f-max**. La moitié de la fréquence d'échantillonnage, la fréquence de Nyquist, est la plus haute fréquence qu'on peut représenter sans ambiguïté. En dessous, on a du repliement de spectre, l'aliasing : des fréquences hautes se déguisent en basses fréquences parasites. Concrètement : pour le son du moteur, le contenu utile est sous 1 kHz, donc 16 kHz d'échantillonnage est largement suffisant. Et pour la vibration, la signature est sous 5 Hz, donc nos 10 Hz effectifs respectent le critère, malgré le débit Bluetooth limité.

Slide 12 — Acquisition des données son/vibration

[6:45]

Voici le diagramme de séquence de l'acquisition. Le micro I2S et le capteur de vibration alimentent l'ESP32 ; celui-ci met en forme les trames SOUND et VIB, les envoie en UDP au client Python, qui les écrit dans des fichiers son et vibration séparés. C'est le cœur de la capture, en temps réel.

Slide 13 — ESP32 : le rôle de concentrateur

[7:15]

L'ESP32 joue le rôle de concentrateur. Un seul microcontrôleur orchestre toute l'acquisition embarquée : il pilote l'interface I2S pour le son, ouvre une connexion Bluetooth cliente vers le capteur de vibration, et émet les deux flux en UDP vers le PC. Un mutex FreeRTOS sérialise les envois pour éviter que les paquets son et vibration ne s'entremêlent. Nous avons aussi optimisé le firmware : snprintf à la place de String, activation de l'APLL, buffers DMA multipliés par huit — c'est ce qui nous a fait passer de 10 à près de 16 kHz effectifs sur le son.

Slide 14 — Architecture logicielle bout-en-bout

[8:00]

Voici la chaîne complète, de bout en bout : des capteurs vers l'ESP32, puis le Wi-Fi, les serveurs Python, les fichiers CSV, et enfin le modèle d'IA en Keras. Chaque maillon est fonctionnel.

Slide 15 — Plan d'expérience : 110 sessions

[8:30]

Pour constituer le dataset, nous avons défini un plan d'expérience croisé : onze niveaux de tension, de 0 à 100 % par pas de 10 %, multipliés par deux conditions de charge, multipliés par cinq captures de quinze secondes — soit 110 sessions prévues, environ une heure cinquante de banc. Réalisé en deux phases pour limiter la dérive thermique du moteur, avec une vérification du dataset à mi-parcours.

Slide 16 — Pourquoi la tension et non la puissance ?

[9:15]

Un choix important : nous prédisons la tension, pas la puissance. Pourquoi ? La puissance, c'est $P = U \times I$ — il faudrait mesurer simultanément la tension **et** le courant, or nous n'avons pas de capteur d'intensité. Plutôt qu'inventer une mesure indirecte fragile, nous assumons la tension comme proxy : à charge fixée, elle reste une fonction monotone de la puissance. C'est donc un proxy honnête, suffisant pour caractériser l'effort du moteur. Nous travaillons en relatif par condition — chaque condition a sa propre

tension max à 100 % — et le label est normalisé entre 0 et 1 pour le réseau.

Slide 17 — Partie 3 : Ma contribution individuelle [10:00]

J'arrive à ma contribution personnelle : le pipeline d'acquisition logicielle et le modèle d'intelligence artificielle.

Slide 18 — Périmètre de ma contribution

[10:10]

J'ai développé deux briques complémentaires. D'un côté, le pipeline d'acquisition : un serveur TCP de métadonnées sur le port 9090, un client UDP de données capteurs sur le port 9091, et le format du fichier CSV maître. De l'autre, le modèle d'IA : une architecture CNN à deux branches, son et vibration, avec des mel-spectrogrammes en entrée, des augmentations, et un découpage anti-fuite. Les avoir développées en parallèle m'a permis de garantir que les formats produits en amont correspondent exactement à ce qu'attend le modèle en aval.

Slide 19 — Serveur TCP de métadonnées (port 9090)

[10:50]

Le serveur TCP est le point d'entrée des métadonnées. À chaque capture, l'IHM ouvre une connexion courte, envoie un marqueur START suivi d'une ligne CSV de cinq champs — condition de charge, durée, fréquences son et vibration, et niveau de tension. Le serveur attribue un identifiant de session, horodate, normalise la condition de charge via un dictionnaire pour aboutir aux valeurs canoniques A_vide ou B_frein, et écrit la ligne dans le fichier maître.

Slide 20 — Client UDP de données (port 9091)

[11:30]

Le client UDP reçoit les signaux à haut débit. J'ai agrandi le buffer de réception à 8 Mo pour éviter les pertes liées au débit audio. Il récupère l'identifiant de session courant depuis le fichier maître, détecte automatiquement le début et la fin d'une capture grâce à un timeout d'inactivité de trois secondes, et parse deux formats de trames : SOUND pour l'audio et VIB pour la vibration. Chaque trame vibration est validée strictement à quinze champs ; toute trame mal formée est rejetée — c'est une défense en profondeur contre la corruption des fichiers.

Slide 21 — Format du fichier maître CSV

[12:15]

Le fichier maître `metadata_captures_moteur.csv` est la base de données du dataset. Neuf colonnes : l'identifiant, l'horodatage, la condition, la durée, les deux fréquences, les noms des fichiers son et vibration, et le niveau de tension — le label. C'est un véritable contrat de données, partagé entre l'IHM, l'acquisition et le modèle. Avantage : à l'apprentissage, tout le dataset se charge en un seul `read_csv`, sans logique de découverte de fichiers.

Slide 22 — Pourquoi une architecture double branche ?

[12:55]

Pour le modèle, le point clé est le choix d'une architecture double branche. Le son et la vibration ont des natures physiques très différentes : l'un échantillonné à plusieurs kilohertz, riche en hautes fréquences ; l'autre à quelques hertz, porteur d'une signature basse fréquence. Les fusionner dans un seul tenseur obligerait le réseau à apprendre deux statistiques très distinctes en même temps, ce qui nuit à la convergence. La solution : une branche convolutive dédiée à chaque modalité, puis une fusion tardive par concaténation. Avantage supplémentaire : on peut désactiver une branche pour mesurer sa vraie contribution.

Slide 23 — Modèle IA (diagramme)

[13:40]

Voici le diagramme de séquence côté inférence : l'opérateur saisit les données dans l'IHM, qui les envoie au serveur Python ; celui-ci appelle le modèle, qui renvoie le niveau estimé, affiché à l'opérateur. La boucle complète, de la saisie à la prédiction.

Slide 24 — Transformée de Fourier

[14:10]

Avant de montrer comment je transforme les signaux, un deuxième fondement théorique : la transformée de Fourier. L'idée est de passer du domaine temporel au domaine fréquentiel pour révéler le contenu spectral. La transformée de Fourier décompose un signal en une somme de sinusoides et donne l'amplitude de chaque fréquence présente — c'est la formule **X de f égale l'intégrale de x de t fois e puissance moins**

deux i p i f t. En pratique, on utilise la version à court terme, la STFT : on applique la transformée de Fourier rapide sur des fenêtres glissantes, ce qui permet de suivre l'évolution du spectre dans le temps — c'est ça, un spectrogramme. On passe ensuite à l'échelle de Mel, une compression perceptuelle calquée sur l'oreille humaine, plus fine dans les basses fréquences. Puis conversion en décibels et normalisation, ce qui donne une image exploitable par le réseau.

Slide 25 — Représentation : mel-spectrogrammes

[15:05]

Concrètement, voici le pipeline de transformation, qui reprend ce qu'on vient de voir : signal temporel, STFT, passage à l'échelle Mel, log en décibels, normalisation entre 0 et 1, et redimensionnement à 128 par 128 pixels. Je découpe le son en fenêtres d'une seconde avec 50 % de recouvrement — une augmentation gratuite des données. Le mel-spectrogramme du son a 64 bandes, celui de la vibration 32, avec des paramètres réduits. Toutes les entrées du réseau sont ramenées à la même géométrie : 128 par 128, un seul canal.

Slide 26 — Augmentations et split anti-fuite

[15:50]

Avec un dataset modeste, deux précautions. D'abord, des augmentations, uniquement sur l'entraînement : bruit gaussien, décalage temporel, et SpecAugment — du masquage de bandes de temps et de fréquence — ce qui multiplie virtuellement la taille du dataset par environ quatre. Ensuite, un découpage anti-fuite par capture : toutes les fenêtres d'une même capture vont dans le même ensemble, train, validation ou test. Cela empêche le modèle de mémoriser une signature vue à l'entraînement, et l'évaluation se fait par capture, pas par fenêtre — c'est plus honnête.

Slide 27 — Architecture du réseau de neurones

[16:35]

Voici l'architecture détaillée. Chaque branche enchaîne trois blocs de convolution — 32, 64, 128 filtres — suivis d'un global average pooling. Les deux branches sont concaténées, passent par une couche dense de 64 neurones avec dropout, puis une sortie dense unique avec une sigmoïde multipliée par 100, qui donne le pourcentage. C'est une régression : fonction de coût MSE, métrique MAE en pourcentage, optimiseur Adam, 50 epochs maximum avec arrêt anticipé.

Slide 28 — Décision d'ingénierie · Option A : |VZ| seul

[17:20]

Une dernière décision d'ingénierie, que j'aime présenter car elle est défendable. Sur les trois axes de vibration, j'ai observé que VX et VY étaient figés à 271 et 1 dans notre montage. Après vérification avec le manuel du constructeur, j'ai identifié la cause : ces axes sont perpendiculaires à la vibration dominante du moteur, donc sous le seuil de bruit du capteur. Or, prendre la magnitude trois axes aurait introduit un plancher à 271 qui écrase la dynamique utile. J'ai donc retenu uniquement la valeur absolue de VZ. Ce n'est pas un défaut hardware ignoré : c'est un choix justifié par une analyse statistique et la documentation constructeur.

Slide 29 — Résultats · courbes d'entraînement

[18:05]

Voici les courbes d'entraînement. La convergence est stable sur les epochs, sans divergence significative entre l'entraînement et la validation. La MAE d'entraînement décroît régulièrement, et la validation se stabilise — ce qui montre qu'on a bien atteint la fin de l'apprentissage utile, sans surapprentissage marqué.

Slide 30 — Résultats · MAE 11,2 % par capture

[18:35]

Le résultat principal : une erreur absolue moyenne de 11,2 % par capture, exprimée directement en pourcentage de tension — une métrique métier interprétable. Pour situer : une baseline naïve qui prédirait toujours la moyenne ferait environ 27 % d'erreur. Notre modèle divise donc l'erreur par environ 2,5. C'est une preuve de concept fonctionnelle, avec une vraie marge de progression : une campagne complète et une meilleure qualité de vibration. Je l'assume comme un proof-of-concept défendable, sur un dataset volontairement limité.

Slide 31 — Conclusion et perspectives

[19:10]

Pour conclure. Sur le plan technique, nous avons une chaîne complète et fonctionnelle, de bout en bout, un modèle d'IA défendable à 11,2 % de MAE, et des décisions d'ingénierie documentées à chaque étape. Côté perspectives : compléter la campagne des 110 captures, ajouter un capteur de courant pour passer du proxy

tension à la mesure directe de la puissance, et comparer les performances mono-modalité contre double branche. Je vous remercie pour votre attention, et je suis prête à répondre à vos questions. [~19:45]

Notes de présentation

- **Minutage** : viser ~19-20 min. Si tu dépasses, les slides à raccourcir en priorité sont les diagrammes de séquence (12 et 23) et l'architecture logicielle (14) — une phrase suffit.
- **Formules** : à l'oral, énonce-les en mots (comme écrit ci-dessus) ; ne lis pas les symboles un par un.
- **Slides théoriques (11 Shannon, 24 Fourier)** : ce sont tes points forts pour le jury — montre que tu comprends *pourquoi* tes choix techniques tiennent. Ne les survole pas.
- **Questions probables du jury** : SR effective 10,7 → 15,8 kHz, conflit radio BLE/Wi-Fi, pourquoi 59 captures au lieu des 110 prévues, interprétation de la MAE, repliement de spectre, différence régression/classification.